

Solution

2024 CCPC 东北四省赛

题解



May 19, 2024

J. Breakfast

题意

一个包子 0.6 元。一个鸡蛋 1 元。

问早饭买 n 个包子和 m 个鸡蛋最少需要多少钱。

$n = 32$ 、 $m = 20$ 。

J. Breakfast

题解

输出 39.20 即可。

D. nIM gAME

题意

一堆 n 个石子，两人轮流拿，每次只能拿 $1 \sim 3$ 个。

假设整个游戏过程中先手拿了 $a_1, a_2, \dots, a_k$ 个，那么他能赢当且仅当

$$a_1 \oplus a_2 \oplus \dots \oplus a_k = 0。$$

问先手能不能赢。

D. nIM gAME

题解

手玩发现 $n = 1$ 到 6 均为必败，此时可以猜结论：先手必败。胆子大一些直接就过了（

考虑证明， $n > 6$ 时，经过先后手若干次操作，Mandy 一定能拿到这样一个局面： $n = 3 \sim 6$ ，且 brz 之前的操作异或和为 $0 \sim 3$ 。证明显然。

而通过打表或者手玩可以发现，这 16 种情况无论遇到哪种，Mandy 都可以操作使得 brz 必败。所以 brz 永远必败。

主要考察点可能是做题策略和简单 dp 进行小范围打表。

E. Checksum

题意

给你一个 n 位二进制数 A ，需要构造一个最小的长度为 k 的二进制数 B ，使得 $B \equiv \text{popcount}(A) + \text{popcount}(B) \pmod{2^k}$ 。
 $n \leq 10^5$, $k \leq 20$ 。

E. Checksum

题解

枚举，信息串 C 中 1 的个数是 A 和 B 的 1 的个数总和，而 A 固定， B 中 1 的个数只可能是 $0 \sim k$ ，所以只需要枚举 B 中 1 的个数，然后暴力验证即可。

A. Paper Watering

题意

给你一个数 n ，最多进行 k 次操作。每次操作可以把这个数开根号下取整，或者平方。问最多 k 次操作后可以得到的不同的数的个数。
 $n, k \leq 10^9$ 。

A. Paper Watering

题解

假设一直进行操作 1 (开根), 得到的数字依次为 $a_0, a_1, a_2, \dots, a_m$ (其中 $a_0 = x$), 那么对于满足 $a_i^2 \neq a_{i-1} (i > 1)$ 的 i , 会产生贡献 $k - i + 1$, 因为开根到 a_i 时, 花费了 i 次操作, 剩下还能平方 $k - i$ 次, 得到的数字一定都是之前没有出现过的, 算上自己就一共有 $k - i + 1$ 个。

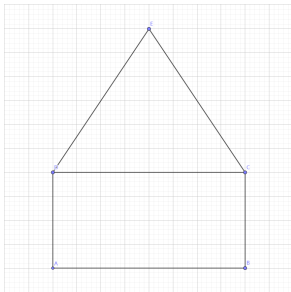
否则 (也就是 $a_i^2 = a_{i-1}$) 只产生 1 的贡献, 也就是只有自己是没有出现过的, 而自己平方若干次能得到的数, 一定是 a_{i-1} 平方若干次能得到的数的子集, 所以没有新的贡献。

另外 a_m 如果为 1, 它也只能产生 1 的贡献而不是 $k - m + 1$, 因为 1 平方多少次都等于自己。

M. House

题意

二维平面上 n 个整点，问选出 5 个不同的点形成房子的方案数。
 $n \leq 300$ 。

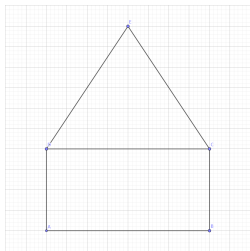


M. House

题解

枚举射线 DC ，然后再枚举第三个点 X 。

- 1 如果在 DC 线上，则跳过。
- 2 如果第三个点在射线 DC 左侧，则看一下 X 到 C 、 D 两点的距离是否相等。
- 3 如果第三个点在射线 DC 右侧，则判断 XD 、 XC 是否与 DC 垂直。如果满足其中任意一个，则分别使用数据结构去维护一下到 D 或 C 的距离。



统计一下第三种情况中，到 D 、 C 两点距离相同的点对即可。

时间复杂度 $O(n^3)$ 或 $O(n^3 \log n)$ 都可通过。

实际上，给你 n 个二维平面上的点，最多只能构造出 $O(n^{2.5})$ 个矩形。

因此，只要在 $O(n^5)$ 或 $O(n^4)$ 等暴力中加入适当剪枝，就可以通过此题。

H. Meet

题意

有一棵大小为 n 的树，和 m 个点对，每个点对中的两点都有对应的 LCA 。
现在让你选择一个点作为根节点，使得所有 $2m$ 个点到它们对应 LCA 的最大距离最小，输出这个数值。

$n, m \leq 10^5$ 。

H. Meet

题解

首先二分答案，然后考虑怎么 check。

不妨暂且将 1 作为根，记当前二分的答案为 mid 。

那么对于一对点 x, y ，记两者的距离为 dis ，若 $dis \geq mid$ ，则可以选任何一个点为根。

否则考虑它们到 lca 的距离：

记 x 到 lca 的距离为 len 。对于 x 来说：

- 若 $len > mid$ ，根必须在 x 的 mid 级祖先对应的子树内。
- 若 $len \leq mid$ ，根必须在 y 的 $dis - mid - 1$ 级祖先对应的子树外。

对于 y 来说同理。

这样每个点对应的合法选根方案都是 dfs 序上的 $O(1)$ 段区间，最后看交集是否非空即可。

时间复杂度 $O(n \log n)$ 或 $O(n \log^2 n)$ 。

I. Password

题意

一个长度为 n 的序列 a ，每一项都是 1 到 k 之间的正整数。如果一个区间 $[i, i + k - 1]$ 是 1 到 k 的排列，那么说明这个区间的所有位置都是好的。问有多少种序列满足所有位置都是好的。

$n \leq 10^5, k \leq 10^3$ 。

I. Password

题解

不妨设 f_i 为最后一个完整排列的结尾是 i 的方案数。于是可以列出转移式：

$$f_i = \sum_{j=1}^k f_{i-j} \times g_j$$

g_j 即在一个 $1 \sim k$ 的排列后接上 j 个数，使得满足以下两个条件的方案数：

- $[j+1, j+k]$ 是一个 $1 \sim k$ 的排列，
- 对于任意 $1 < i \leq j$, $[i, i+k-1]$ 不是一个 $1 \sim k$ 的排列。

直接拿 $1, 2, 3 \cdots k$ 来考虑 g_j 怎么求，那么即要求一个 $1 \sim j$ 的排列，对于任意 $i < j$ ，这个排列 $[1, i]$ 的前缀位置上不能是一个 $1 \sim i$ 的排列，求满足条件的排列个数。

考虑容斥，首先令 $g_j = j!$ ，然后考虑减去不合法的，对于一个不合法的排列，它可能存在若干个前缀符合 $[1, i]$ 是一个 $1 \sim i$ 的排列，那么我们枚举每一个不合法排列最后一个违反限制的前缀，在这个位置将其减去。

I. Password

题解

假设当前枚举到 i , 首先 $[1, i]$ 这部分肯定是 $i!$ 种填法, 而 $[i+1, j]$ 这部分, 由于我们钦定 i 是最后一个违反限制的前缀, 故 $[1, i]$ 与 $[i+1, j]$ 相接不可再违反限制, 即对于任意 $i < p < j$, $[i+1, p]$ 这一段上不能是将 $i+1 \sim p$ 这些数任意排列的结果, 于是就变成子问题了, 乘上 g_{j-i} 就好了。

所以就是两个简单的式子:

$$\blacksquare g_i = i! - \sum_{j=1}^{i-1} j! \times g_{i-j}$$

$$\blacksquare f_i = \sum_{j=1}^k f_{i-j} \times g_j$$

时间复杂度 $O(nk)$ 。

K. Tasks

题意

有 i 个区间 $[l_i, r_i]$ ，每个区间有一个烦躁值 b_i 。

要求每个区间的烦躁值等于所有包含它的区间的最大烦躁值 $+1$ （如果没有包含它的区间，则它的烦躁值为 0 ）。

现在给定所有区间的左端点 l_i ，要求构造所有的右端点 r_i 满足上面的限制（如果无解输出 -1 ）。

$n \leq 10^5, 1 \leq l_i \leq r_i \leq 10^6, 0 \leq b_i \leq 10^6$ 。

K. Tasks

题解

考虑从 $b_i = 0$ 开始构造，把 b_i 相同的任务看成同一层。

不难发现，若存在两个任务， l 和 b 都相等，则无解。

在同一层，限制只有，若 $l_i < l_j$ 则 $r_i < r_j$ 。

现在考虑上一层对于下一层的限制。

对于 l_i ，找到上一层最靠后的 l_j 满足 $l_j \leq l_i$ 。

若 $l_j = l_i$ ，则 $r_i < r_j$ ，否则 $r_i \leq r_j$ 。

故不妨从 l_i 最靠后的任务开始确定 r_i ，取到最右的可行位置。

这样可以使得下一层的任务可以选择的位置尽可能多。

时间复杂度 $O(n \log n)$ 。

L. Bracket Generation

题意

给你一个长度为 n 的合法的括号序列 A ，问有多少种不同的操作序列使得 $()$ 变为 A ，每次可以：

- 1 在序列最右边添加一对括号，即 $S \rightarrow S()$
- 2 选取一个 S 的区间 $[l, r]$ ，要求这个区间内的括号是一个合法的括号序列，然后在这个区间外增加一对括号将其括在其中。

$n \leq 10^6$ 。

L. Bracket Generation

题解

首先可以将括号序列建成一棵有根树，根节点是个虚根，括号里直接包含的括号对就是儿子节点，儿子从左到右排列，保持在括号序列里的相对顺序。注意到叶子节点对应一个操作 1，非叶子节点对应一个操作 2，最后的操作序列可以看做节点编号的一个排列，表示节点的生成顺序，同时有一些限制：

- 1 一个非叶子节点必须在他子树内所有叶子节点都生成完之后才能生成。
- 2 由于节点的儿子是有序的，所以叶子节点也是有序的，每个叶子节点都必须等它左边的所有叶子节点生成完了之后才能生成。

L. Bracket Generation

题解

换个角度看，当我们对这棵树进行先序遍历得到 A 序列后，假设 A 的一个后缀 $A_i \sim A_n$ 能形成若干种不同的操作序列，对于 A_{i-1} ，假如是个叶子结点，那么只能放到操作序列的开头；假如是个非叶子结点，则可以插入到操作序列中的任意位置。

于是记一个 B 序列， A_i 是叶子结点时， $B_i = 1$ ，否则 $B_i = n - i + 1$ ，

$\prod_{i=1}^n B_i$ 就是答案。

另外，注意到如果树中存在一条链，也就是存在 $(((\cdots)))$ 形式的括号序列，那么它们插入到操作序列是需要考虑一些组合排列问题的，但是简单推一推柿子后可以发现和上面描述是等价的，所以上面的答案是正确的。

F. Factor

题意

给你 p, x, k , 计算在 1 到 x 的范围内, 有多少个整数 q 使得 $\frac{p}{q}$ 在 k 进制下是一个有限小数 (含整数)。

$1 \leq p, x \leq 10^{14}, 2 \leq k \leq 10^{14}$.

F. Factor

题解

对于一个 $q \in [1, x]$, $\frac{p}{q}$ 在 k 进制下为一个有限小数当且仅当 $q|p \times k^\infty$ 。

也就是说：将 q 进行质因数分解后，其质因子 p_i 及其幂 a_i 要满足以下任一条件：

1 $p_i|k$;

2 $p_i|p$ 且 p_i 的幂次 a_i 小于等于 p 的幂次。

可以对于所有满足 $p_i|k$ 或 $p_i|p$ 的因子。按照如上限制枚举其幂次，然后 dfs。答案的个数约为 10^8 ，也可通过。

或者按照上述条件分开枚举质因子，然后合并，速度会稍快。时间复杂度主要体现在枚举只包含 k 的质因子中。

G. Diamond

题意

给一个长度为 n 的序列 a 。

每次询问指定 l, r, p, q 四个参数，表示只保留下标在 $[l, r]$ 范围内且值为 p 或者 q 的数，其它全部删除。

求剩下序列的逆序数。

$n \leq 10^5$ 。

G. Diamond

题解

如果在一个区间中， p 、 q 两个数的个数都比较小，那么就可以进行暴力。这启发我们考虑对出现次数进行根号分治，如果当前询问的两个参数 p, q 出现次数均小于等于 $\sqrt{n}$ 那么直接暴力即可，单组询问时间复杂度 $O(\sqrt{n})$ 。否则，肯定存在一个出现次数超过 $\sqrt{n}$ ，我们令次数超过 $\sqrt{n}$ 的颜色为关键颜色。会发现这样的颜色总数不超过 $\sqrt{n}$ 。考虑这 $\sqrt{n}$ 个关键颜色与所有颜色间的答案。为方便讲述不妨令关键颜色的值大于所有其他颜色，那么对于每个点我们可以记录一些当前点前面有多少个关键点。具体地不妨设 cnt_i 表示 i 号点前面有多少个关键颜色，那么对于一组询问 l, r, p, q 其中 q 为关键颜色答案就是统计 $[l, r]$ 中下标颜色是 p 的 cnt 之和，这个可以记录前缀和得到，对于单个颜色时间复杂度为 $O(n)$ 。总时间复杂度为 $O((n + q)\sqrt{n})$ 。

C. Ring

题意

n 颗珠子围成一个环，每颗珠子的颜色为红色或者蓝色。

每次操作可以选定任意连续的 k 个珠子，并把它们全部变为这 k 个珠子中较多的颜色。

求将所有 n 个珠子颜色全部统一的最小操作数，以及能够满足最小操作数的所有操作方案中，第一次操作的位置。

$n \leq 10^6$ 。

C. Ring

题解

首先枚举第一步。假设第一步使得这个区间变成了 x ，那么：

Lemma

引理 1 以后所有的操作都将变成 x ，否则不优。

证明：如果最后所有数变成了 x ，那么以后的操作变为 $1 - x$ 不优；如果最后所有数变成的是 $1 - x$ ，那么第一步操作不优。

Lemma

引理 2 枚举第一步后，从一个方向开始一直操作，直到第一个不是按顺序操作的位置。然后把这个不按顺序操作的操作改为按顺序操作一定更优。

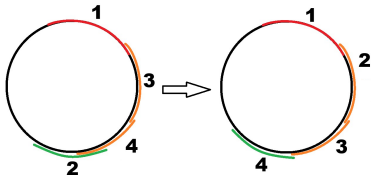
C. Ring

题解

Lemma

引理 2 枚举第一步后，从一个方向开始一直操作，直到第一个不是按顺序操作的位置。然后把这个不按顺序操作的操作改为按顺序操作一定更优。

证明：考虑左图，依次操作 1, 3, 4，然后发现 2 已经在之前已经被操作过了，那么我们可以把 2 挪到 4 之后再操作，变成右图的顺序，肯定不劣。对于其余情况，也可以用类似方法证明。



C. Ring

题解

Lemma

引理 3 一定存在一种操作次数最少的方案，使得其一直沿着顺时针或逆时针方向操作。

在一个方向上，一直使用引理 2，即可得到引理 3。

Lemma

引理 4 顺时针操作的最优步数 = 逆时针操作的最优步数。

由引理 2, 3，我们可以证明顺时针步数 $\leq$ 逆时针步数，同理可证逆时针步数 $\leq$ 顺时针步数。因此两者相等。

C. Ring

题解

所以。我们可以枚举第一步，假设变为 x 。然后从任一方向开始模拟：

- 每次往该方向选择一个可以令最多的 $1 - x$ 变成 x 的区间，然后操作。直到所有数变成 x 。

记录 $f_{i,0/1}$ 表示如果区间 $[i, (i + k - 1) \bmod n]$ 变成 0/1 后，下一步操作该操作哪里、以及下一步能消掉多少个 1/0。

这个可以 two-pointer 模拟，时间复杂度 $O(n)$ 。

枚举第一步操作了 i ，然后使用倍增判断还需要操作多少次，时间复杂度 $O(n \log n)$ 。

实现上有若干细节。总的复杂度为 $O(n \log n)$ 。

B. Charging Station

题意

$3n$ 个供能站，每级 n 个，

每个供能站可能有三种状态：不工作，供能，辅助供能。

供能模式供能，辅助功能模式吸能。

供能站之间有一些限制关系，每个供能模式的供能站需要下一级的一些供能站吸能。

问最大总供能是多少。

$n \leq 10^5$ 。

B. Charging Station

题解

假如只有两级，只需要先假设低级的供能站都处于辅助供能状态，那么是一个经典的网络流最大权闭合子图问题。

变成三级之后，看似是三层的需求关系，实际上是一和三级的充能站共同对二级有需求，还是可以转化为一个两层的需求关系。

显然一级只能是不工作或供能状态，三级只能是供能或辅助供能状态，但是二级三种状态都有可能，还需要进行一些转化。

假设二级一开始是工作状态，则一级一开始是不工作状态，三级一开始是辅助供能状态。注意到三级的供能站想要转为供能状态，则需要二级处于不工作状态；一级供能站想要转为供能状态，则需要二级处于辅助供能状态。

B. Charging Station

题解

于是可以将二级拆点，拆成供能转不工作和不工作转辅助供能两个状态，代价分别为 $-a_i$ 和 $-b_i$ 。

然后有需求的三级供能站只向前者连边，一级供能站同时向两者连边，流量均为无穷，保证跑最小割时割不掉。

然后跑最大权闭合子图即可。复杂度 $O(m\sqrt{n})$ 。

